# Implement a custom `reduce()` (Array.prototype polyfill)

**Company:** Amazon · **Tech stack:** Vanilla JavaScript (Node / browser) · **Time limit:** 60 minutes (default)

> Write `Array.prototype.myReduce(callback, initialValue)` that behaves exactly like the native `Array.prototype.reduce`.

---

## PART A: Thought process

### General steps for any machine coding task

1. **Understand the requirements:** restate the problem, separate functional from non-functional, ask clarifying questions, write down assumptions.
2. **List interactions / use cases:** every input and its expected output. This table is your test list.
3. **Design the architecture:** the pieces and who does what. Start with one unit; split only when a responsibility is clear.
4. **Design the state / data model:** what has to be remembered while the code runs, and what can be derived.
5. **Define the contracts:** function signatures, inputs, outputs, errors.
6. **Trace the data flow** for one key case end to end.
7. **Find the hard parts** before coding, so you don't discover them at minute 50.
8. **Build in layers:** simplest working version first, then add one correct layer at a time, running it after each.
9. **Cover states and edge cases.**
10. **Wrap up:** explain trade-offs and what you'd add with more time.

### Time budget (60 minutes)

| Phase | Time |
|---|---|
| Requirements, clarifying questions, assumptions | 5 min |
| Use cases + edge-case list (written as test calls) | 5 min |
| Design: signature, seeding rule, loop rule | 5 min |
| Basic version (happy path, with initial value) | 10 min |
| Layers: no-seed, holes, guards, prototype attach | 15 min |
| Run tests against native `reduce`, fix diffs | 10 min |
| Wrap-up + follow-ups (map/filter via reduce, async reduce) | 10 min |

### Think out loud

The interviewer grades the reasoning, not just the final code. Say every decision out loud: "*I'm checking `arguments.length` instead of `initialValue === undefined` because someone can pass `undefined` on purpose.*" If you get stuck, they can only help you if they know where you are.

---

## PART B: Apply it to this problem

## 1. Understand the requirements

### Functional requirements

- `arr.myReduce(callback, initialValue?)` folds the array into one value, left to right.
- The callback receives `(accumulator, currentValue, currentIndex, array)`.
- **With** an initial value: the accumulator starts as `initialValue`, iteration starts at index 0.
- **Without** an initial value: the accumulator starts as the first *present* element, iteration starts after it.
- Empty array (or all holes) with no initial value → `TypeError: Reduce of empty array with no initial value`.
- Non-function callback → `TypeError`.
- Returns the final accumulator.

### Non-functional requirements

| Concern | What it means here |
|---|---|
| Spec correctness | Same output and same errors as native `reduce` for every input, including odd ones. |
| Performance | One pass, O(n) time, O(1) extra space. A plain `for` loop, no recursion, no copies. |
| Error handling | Throw the same `TypeError`s as native, before doing any work. |
| Safety | Don't break other code: non-enumerable property, don't overwrite an existing method. |
| Reusability | Works on arrays, array-likes (`arguments`, NodeList, `{length}` objects) and strings via `.call`. |
| Extensibility | Same skeleton gives `reduceRight`, `map`, `filter`, async reduce. |

### Clarifying questions to ask

1. Should it be on `Array.prototype` (polyfill) or a standalone `reduce(arr, fn, init)` function?
2. Must it match the spec on sparse arrays (holes), or only dense arrays?
3. Should passing `undefined` explicitly as the initial value count as "an initial value was given"?
4. Must it work on array-likes and strings via `Array.prototype.myReduce.call(...)`?
5. What should happen if the callback mutates the array during iteration (push / delete)?
6. Do you also want `reduceRight` or an async version as a follow-up?

### Assumptions we will build with

- It is a polyfill on `Array.prototype` named `myReduce` (avoid clobbering the native one in the interview).

- It follows the ECMAScript algorithm: holes are skipped, `length` is read once, explicit `undefined` is a real seed.
- It works on any array-like through `.call`.
- No `thisArg` (native `reduce` doesn't have one; the callback runs with `this === undefined`).

---

## 2. Identify interactions / use cases

| Action / input | Expected behavior / output |
|---|---|
| `[1,2,3,4].myReduce((a,b)=>a+b)` | `10`; callback called 3 times, first call `(1, 2, 1, arr)` |
| `[1,2,3,4].myReduce((a,b)=>a+b, 10)` | `20`; callback called 4 times, first call `(10, 1, 0, arr)` |
| `[].myReduce(fn, 0)` | `0`; callback never called |
| `[].myReduce(fn)` | throws `TypeError: Reduce of empty array with no initial value` |
| `[5].myReduce(fn)` | `5`; callback never called |
| `[ , 1, , 2].myReduce(sum)` | `3`; holes skipped, seed is `1` (index 1) |
| `[ , , ,].myReduce(sum)` | throws `TypeError` (no present element) |
| `[1,2].myReduce(sum, undefined)` | `NaN` (`undefined + 1 + 2`): explicit `undefined` is a seed |
| `[1,2].myReduce(123)` | throws `TypeError: 123 is not a function` |
| `Array.prototype.myReduce.call({0:'a',1:'b',length:2}, sum, '')` | `'ab'` |
| `Array.prototype.myReduce.call('xyz', sum, '')` | `'xyz'` |
| `Array.prototype.myReduce.call(null, sum)` | throws `TypeError` |
| callback pushes to the array each step | new items are **not** visited (length read once) |
| callback deletes a later index | deleted index is **skipped** |
| `for (k in [1])` | does not list `myReduce` |

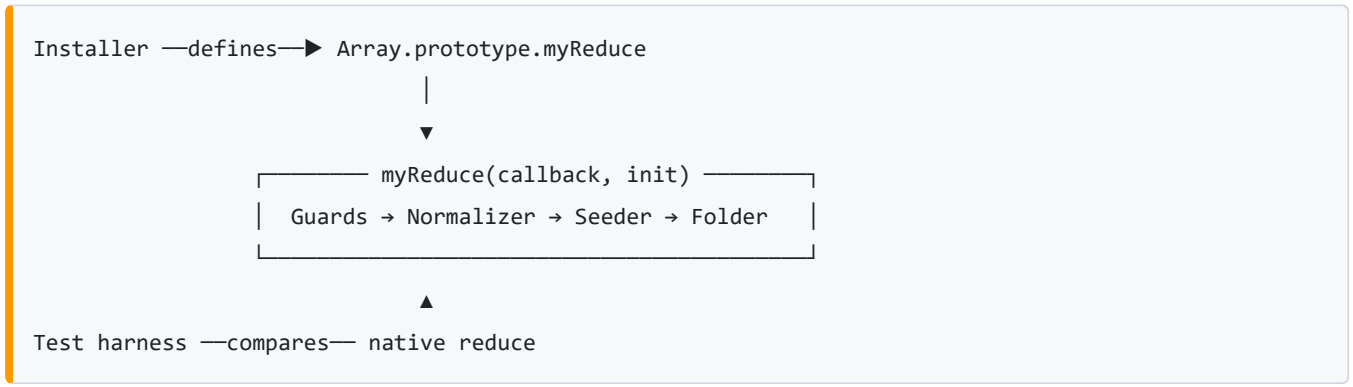
---

## 3. Design the architecture

There is no UI, so the "components" are the steps of one function. Keep it one function; the steps are just clear blocks inside it.

| Piece | Responsibility |
|---|---|
| **Guards** | Reject `this == null` and a non-callable callback with `TypeError`. |

| Piece | Responsibility |
|---|---|
| **Normalizer** | `Object(this)` + `length >>> 0`: turn any receiver into an indexable object with a fixed length. |
| **Seeder** | Decide the starting accumulator and starting index (with or without initial value). |
| **Folder (loop)** | Walk `k → len`, skip holes, call the callback, update the accumulator. |
| **Installer** | Attach `myReduce` to `Array.prototype` as non-enumerable, only if missing. |
| **Test harness** | Run every use case on both `myReduce` and native `reduce`, assert they match. |




---

## 4. Design the state / data model

The "state" is the handful of variables the function holds while it runs.

| State / field | Purpose |
|---|---|
| `O` | `Object(this)`: the receiver as an object (boxes strings, keeps array-likes). |
| `len` | Length read **once** at the start. Fixes how far we iterate even if the array grows. |
| `k` | Current index. Shared by the seeder (skipping leading holes) and the loop. |
| `acc` | The running accumulator, returned at the end. |
| `arguments.length` | Whether an initial value was *passed* (not whether it is `undefined`). |

```
const O = Object(this);
const len = O.length >>> 0;
let k = 0;
let acc;
const hasSeed = arguments.length >= 2;
```

**Senior note:**

- *Derived, not stored:* "has an initial value" is derived from `arguments.length`; "is this index present" is derived from `k in O` each step. We never copy the array or build a list of present indices.
- *Ownership:* the caller owns the array; `myReduce` only reads it. The callback owns the accumulator's shape (number, object, array, string). `myReduce` never inspects `acc`.

---

## 5. Design the data structures and contracts

### Example data and why

```
const dense  = [1, 2, 3, 4];              // happy path
const sparse = [, 1, , 2];                // holes: index 0 and 2 are missing, not undefined
const withUndef = [undefined, 1];         // undefined IS present, must be visited
const arrayLike = { 0: "a", 1: "b", length: 2 };
const orders = [                          // realistic Amazon-style data, stable ids
  { id: "o1", category: "books", total: 20 },
  { id: "o2", category: "toys",  total: 15 },
  { id: "o3", category: "books", total: 5 },
];
```

`sparse` vs `withUndef` is the key pair: a hole and an `undefined` value look the same with `O[k]`, but `k in O` tells them apart.

### Contract

```
myReduce<T, U>(
  callback: (acc: U, cur: T, index: number, array: T[]) => U,
  initialValue?: U
): U
```

| Input | Output |
|---|---|
| valid callback + array (or seed) | final accumulator |
| `this` is `null` / `undefined` | `TypeError` |
| callback not a function | `TypeError: <x> is not a function` |
| no seed and no present element | `TypeError: Reduce of empty array with no initial value` |

Example:

```
orders.myReduce((acc, o) => {
  acc[o.category] = (acc[o.category] ?? 0) + o.total;
  return acc;
}, {});
// → { books: 25, toys: 15 }
```

---

## 6. Trace the data flow

Key case: `[ , 1, , 2].myReduce((a, b) => a + b)` (sparse, no seed).

```
call myReduce(sum)              arguments.length = 1
        |
        ▼
Guards: this != null ✓, typeof sum === "function" ✓
        |
        ▼
Normalize: O = the array, len = 4, k = 0
        |
        ▼
Seed (no initialValue):
   k=0: 0 in O? no  → k=1
   k=1: 1 in O? yes → acc = O[1] = 1, k = 2
        |
        ▼
Fold:
   k=2: 2 in O? no  → skip
   k=3: 3 in O? yes → acc = sum(1, 2, 3, O) = 3
   k=4: stop (k === len)
        |
        ▼
return 3
```

---

## 7. Important technical challenges

### 7.1 Detecting "no initial value"

**Problem:** `arr.myReduce(fn, undefined)` and `arr.myReduce(fn)` look identical if you check `initialValue === undefined`. Native treats the first as a real seed: `[1,2].reduce((a,b)=>a+b, undefined)` is `NaN`, not `3`. Also `0`, `''`, `null` are valid seeds, so `if (!initialValue)` is a classic bug.

**Solution:** count the arguments actually passed.

```
if (arguments.length >= 2) {
  acc = initialValue;
} else {
  // seed from the array
}
```

(This is also why `myReduce` must be a `function`, not an arrow function: arrows have no `arguments` and no own `this`. A rest parameter `(...args)` works too.)

### 7.2 Sparse arrays (holes)

**Problem:** `[ , 1, , 2]` has length 4 but only two elements. Native `reduce` never calls the callback for holes, and when there is no seed it seeds from the **first present** element (index 1), not `O[0]` (`undefined`). A naive

`acc = arr[0]; for (i = 1...)` returns `NaN` here.

**Solution:** use `k in O` both when seeding and in the loop. It is `false` for holes and `true` for a stored `undefined`.

```
while (k < len && !(k in O)) k++;
if (k >= len) throw new TypeError("Reduce of empty array with no initial value");
acc = O[k++];

for (; k < len; k++) {
  if (k in O) acc = callback(acc, O[k], k, O);
}
```

### 7.3 Mutation during iteration

**Problem:** the callback gets the array as its 4th argument and can change it. If it pushes every step and we loop on `O.length`, the loop never ends. If it deletes a later index, we'd pass a stale value.

**Solution:** read `len` **once** before the loop (pushed items are never visited), and check `k in O` **every** step (deleted items are skipped). This matches the spec exactly.

```
const len = O.length >>> 0; // fixed upper bound → always terminates
// ...
if (k in O) acc = callback(acc, O[k], k, O); // re-checked live each step
```

### 7.4 Installing it safely on the prototype

**Problem:** `Array.prototype.myReduce = function...` creates an **enumerable** property, so every `for...in` over any array in the app now also yields `"myReduce"`. Blindly assigning can also overwrite a real implementation.

**Solution:** `Object.defineProperty` with `enumerable: false`, guarded by an existence check.

```
if (!Array.prototype.myReduce) {
  Object.defineProperty(Array.prototype, "myReduce", {
    value: myReduce, writable: true, configurable: true, enumerable: false,
  });
}
```

---

## 8. Implement in the correct order

| # | Step | Working after this step |
|---|---|---|
| 1 | Write the use-case table as test calls ( `same(arr, fn, init)` comparing to native) | A failing test list |

| # | Step | Working after this step |
|---|---|---|
| 2 | Standalone function, **always with a seed**: `acc = init; for (i=0;i<len;i++) acc = fn(acc, arr[i], i, arr)` | Happy path with seed |
| 3 | Move it to `Array.prototype` and use `this` (regular `function`) | `[1,2].myReduce(...)` works |
| 4 | Add no-seed branch using `arguments.length` | `[1,2,3].myReduce(sum)` = 6, explicit `undefined` seed correct |
| 5 | Empty / all-holes without seed → `TypeError` | Empty-array errors match native |
| 6 | Skip holes with `k in O` (seed search + loop) | Sparse arrays match native |
| 7 | Read `len` once, `>>> 0` | Mutation cases match, loop always ends |
| 8 | `Object(this)` + `this == null` guard | Works on array-likes and strings via `.call` |
| 9 | Non-function callback guard | Error matches native |
| 10 | Install via `Object.defineProperty`, non-enumerable, if missing | `for...in` is clean |
| 11 | Run the whole suite, compare against native | All green |

## 9. Cover all states and edge cases

| State / case | Input | Behavior |
|---|---|---|
| Initial | function called | guards run first, nothing iterated yet |
| Seeded | `init` passed | `acc = init`, start at `k = 0` |
| Unseeded | no `init` | `acc = first present element`, start after it |
| Processing | loop running | callback called once per present index `< len` |
| Success | loop ends | return `acc` |
| Empty + seed | `[]`, `init` | return `init`, callback never called |
| Empty, no seed | `[]` | `TypeError` |
| All holes, no seed | `[,,,]` | `TypeError` |
| Single element, no seed | `[5]` | return `5`, callback never called |
| Holes | `[,1,,2]` | holes skipped |
| Present `undefined` | `[undefined, 1]` | `undefined` **is** visited |
| Falsy seed | `0`, `''`, `null`, `false`, `undefined` | used as seed |
| Bad callback | `123`, `null`, `{}` | `TypeError` before any work |

| State / case | Input | Behavior |
| --- | --- | --- |
| Null receiver | `.call(null)` | `TypeError` |
| Array-like / string | `.call({length:2,...})`, `.call("xyz")` | works |
| Array grows in callback | `o.push(x)` | new items not visited |
| Array shrinks / delete in callback | `delete o[2]` | deleted index skipped |
| Callback throws | `throw err` | error propagates, no partial result |

---

## 10. Final structure

```
implememt custom reduce()/
├── index.js        # myReduce + prototype install (+ module.exports for tests)
└── index.test.js   # use-case table run against native reduce with node:assert
```

Run with `node index.test.js`. No build step, no dependencies.

---

## 11. Full working code

`index.js`

```
// Polyfill-style custom reduce, following the ECMAScript algorithm for Array.prototype.reduce
function myReduce(callback, initialValue) {
  // 1. Guard: reduce can be .call()-ed on anything, but not on null/undefined
  if (this == null) {
    throw new TypeError("Array.prototype.myReduce called on null or undefined");
  }
  // 2. Guard: callback must be callable
  if (typeof callback !== "function") {
    throw new TypeError(callback + " is not a function");
  }

  const O = Object(this); // box primitives, keep array-likes working
  const len = O.length >>> 0; // length read ONCE, coerced to uint32
  let k = 0;
  let acc;

  // 3. Seed the accumulator. arguments.length, not `initialValue === undefined`,
  //    because reduce(fn, undefined) is a valid explicit seed.
  if (arguments.length >= 2) {
    acc = initialValue;
  } else {
    while (k < len && !(k in O)) k++; // skip holes to find the first real element
    if (k >= len) {
      throw new TypeError("Reduce of empty array with no initial value");
    }
    acc = O[k++];
  }

  // 4. Fold. `k in O` skips holes and elements deleted mid-iteration.
  for (; k < len; k++) {
    if (k in O) {
      acc = callback(acc, O[k], k, O);
    }
  }
  return acc;
}

// Attach non-enumerable so `for...in` over arrays doesn't pick it up
if (!Array.prototype.myReduce) {
  Object.defineProperty(Array.prototype, "myReduce", {
    value: myReduce,
    writable: true,
```

```
    configurable: true,
    enumerable: false,
  });
}

module.exports = { myReduce };
```

index.test.js

```
const assert = require("node:assert/strict");
require("./index");

// Run the same call on myReduce and native reduce; results (or error types) must match
const same = (arr, ...args) => {
  const run = (m) => {
    try { return { v: arr[m](...args) }; }
    catch (e) { return { e: e.constructor.name }; }
  };
  assert.deepEqual(run("myReduce"), run("reduce"));
};
const sum = (a, b) => a + b;

same([1, 2, 3, 4], sum);
same([1, 2, 3, 4], sum, 10);
same([], sum, 0);
same([], sum);                  // TypeError on both
same([, , ,], sum);             // all holes → TypeError
same([, 1, , 2], sum);
same([, 1, , 2], sum, 100);
same([5], sum);                 // single element, callback never called
same([1, 2], sum, undefined);   // explicit undefined seed → NaN
same([1, 2], 123);              // non-function → TypeError
same(["a", "b", "c"], (acc, x, i) => acc + x + i, "");
same([[1, 2], [3], []], (a, b) => a.concat(b), []);

// callback receives (acc, cur, index, array)
const seen = [];
const arr = [10, 20];
arr.myReduce((a, c, i, o) => { seen.push([a, c, i, o === arr]); return a + c; }, 0);
assert.deepEqual(seen, [[0, 10, 0, true], [10, 20, 1, true]]);

// mutation during iteration: pushes not visited, deletes skipped
const m1 = [1, 2, 3];
assert.equal(m1.myReduce((a, c, i, o) => { o.push(99); return a + c; }, 0), 6);
const del = (a, c, i, o) => { if (i === 0) delete o[2]; return a + c; };
assert.equal([1, 2, 3, 4].myReduce(del, 0), [1, 2, 3, 4].reduce(del, 0));

// array-likes, strings, null receiver
const call = (o, ...a) => Array.prototype.myReduce.call(o, ...a);
assert.equal(call({ 0: "a", 1: "b", length: 2 }, sum, ""), "ab");
assert.equal(call("xyz", sum, ""), "xyz");
assert.throws(() => call(null, sum), TypeError);

// non-enumerable
```

```
for (const k in [1]) assert.notEqual(k, "myReduce");

console.log("all reduce checks passed");
```

---

## 12. Wrap-up

### Edge cases handled

- Seeded vs unseeded, including falsy and explicit-`undefined` seeds.
- Empty and all-holes arrays (with and without seed).
- Sparse arrays: holes skipped, present `undefined` visited.
- Array mutated during iteration: growth ignored, deletions skipped, loop always terminates.
- Array-likes and strings via `.call`; `null` / `undefined` receiver rejected.
- Non-function callback rejected before any work.
- Prototype property is non-enumerable and not overwritten if present.

### Trade-offs to mention

- **Prototype polyfill vs standalone function:** the polyfill matches how `reduce` is used, but extending built-in prototypes is risky in shared code (name clashes, the "SmooshGate" `flatten` story). In a real app I'd ship a standalone `reduce(arr, fn, init)`.
- `>>> 0` **vs spec** `ToLength`**:** `>>> 0` (uint32) is the classic polyfill choice and correct for real arrays; the modern spec uses `ToLength` (up to 2^53−1) for huge array-likes. Mention it, don't over-build it.
- **Loop vs recursion:** a recursive fold looks elegant but blows the stack around ~10k elements. The loop is O(1) stack.
- **Error messages:** types match native; exact message text differs between engines, so I match type, not text.

### With more time

- `myReduceRight` (same skeleton, `k` from `len - 1` down to `0`).
- `map`, `filter`, `flat`, `groupBy`, `pipe/compose` built on `myReduce`.
- An async reduce that runs promise-returning callbacks in sequence.
- Property-based tests (random sparse arrays vs native) and a test runner (Jest / `node --test`).
- TypeScript overloads for the seeded / unseeded signatures.

---

## 13. Counter questions the interviewer may ask

### Requirements

#### Q1. Why does native `reduce` throw on an empty array without an initial value, instead of returning `undefined`?

There is no value to start the accumulator with and nothing to return that would be "correct" for every reducer. `undefined` would silently hide bugs (e.g. `sum` of `[]` becoming `undefined`). Throwing forces the

caller to supply the identity ( `0` for sum, `[]` for concat).

### Q2. Does `reduce` take a `thisArg` like `map` and `forEach`?

No. `map/filter/forEach` take `(callback, thisArg)`; `reduce`'s second parameter is the initial value. The callback runs with `this === undefined` (in strict mode). If you need a context, bind the callback or close over it.

### Q3. Can I stop `reduce` early, like `break`?

Not natively. Options: throw and catch (ugly), use `some` / `find` /a `for...of` loop when you need early exit, or build a variant that checks a sentinel ( `if (acc === STOP) break` ). Mention that a plain loop is clearer than forcing early exit into `reduce`.

## State design

### Q4. Why `arguments.length >= 2` and not `initialValue !== undefined`?

Because `undefined` can be passed on purpose and native treats it as a seed: `[1,2].reduce(sum, undefined)` is `NaN`. A truthiness check is worse: `0`, `''`, `null`, `false` are all legitimate seeds.

### Q5. Could you write it as an arrow function?

No. Arrow functions don't have their own `this` (we need the array) or `arguments` (we need the seed check). You could use `function (callback, ...rest)` and check `rest.length`, but it must be a regular function.

### Q6. Why `Object(this)`?

It boxes primitives (so `.call("xyz")` gives a `String` object with indices and `length`) and leaves objects untouched. Combined with the `this == null` guard, it matches the spec's `ToObject(this)`.

## Challenge: holes

### Q7. Why `k in O` instead of `O[k] !== undefined`?

A hole and a stored `undefined` both read as `undefined`. `[undefined, 1].reduce(...)` must visit index 0; `[, 1].reduce(...)` must not. `in` checks presence (own or inherited), which is exactly what the spec's `HasProperty` does.

### Q8. Why `in` and not `hasOwnProperty`?

The spec uses `HasProperty`, which also sees inherited indices. If someone sets `Array.prototype[0] = 'x'`, native reduce sees it for holes at index 0. `hasOwnProperty` would diverge. In practice it rarely matters, but `in` is both shorter and correct.

## Challenge: mutation

### Q9. What happens if the callback pushes into the array?

Nothing new is visited, because `len` was read once before the loop. That is spec behavior and also guarantees termination. If we used `O.length` in the loop condition, `arr.reduce((a, c, i, o) => (o.push(1), a))` would loop forever.

### Q10. And if it deletes or changes a later element?

Changes are seen (we read `O[k]` live when we get there). Deleted indices are skipped because `k in O` is now false. Shrinking `length` behaves the same way: removed indices fail the `in` check.

## Challenge: prototype

### Q11. Why `Object.defineProperty` instead of `Array.prototype.myReduce = ...`?

Assignment makes the property enumerable, so every `for...in` over every array in the app would yield `"myReduce"`. `defineProperty` with `enumerable: false` makes it behave like built-in methods. The existence check avoids overwriting a real implementation.

### Q12. Is extending `Array.prototype` a good idea in production?

Generally no in apps and libraries: name clashes with future standards (the `Array.prototype.flatten` → `flat` rename happened because of MooTools), and hidden global side effects. Polyfills are fine when they exactly follow a standard and are guarded with "only if missing". Otherwise export a function.

## Implementation steps

### Q13. Implement `map` and `filter` using your `reduce`.

```
const map = (arr, fn) => arr.myReduce((acc, x, i, a) => (acc.push(fn(x, i, a)), acc), []);
const filter = (arr, fn) => arr.myReduce((acc, x, i, a) => (fn(x, i, a) && acc.push(x), acc), []);
```

Note `push` into the accumulator, not `[...acc, x]`, to stay O(n).

### Q14. How would you implement `reduceRight`?

Same code, but seed by scanning `k` from `len - 1` down, and loop `for (; k >= 0; k--)`. Error rules and hole handling are identical.

### Q15. Write `pipe` / `compose` with `reduce`.

```
const pipe = (...fns) => (x) => fns.reduce((v, f) => f(v), x);
const compose = (...fns) => (x) => fns.reduceRight((v, f) => f(v), x);
```

### Q16. How would you do an async reduce where the callback returns a promise?

Make the accumulator a promise and chain: `arr.reduce((p, x) => p.then(acc => fn(acc, x)), Promise.resolve(init))`. This runs steps in sequence. Or a plain `async` loop with `await`, which is easier to read and debug.

## Performance

### Q17. What is the time and space complexity?

O(n) time: each index is checked once. O(1) extra space: a few variables, no copies. The callback's own cost is on top; a reducer that does `{...acc, [k]: v}` each step makes the whole thing $O(n^2)$.

### Q18. What if the array has 1 million items?

The loop handles it fine: O(n), constant stack. The danger is the reducer: spreading the accumulator per step (`[...acc, x]`, `{...acc}`) copies on every call, so 1M items becomes ~10^12 operations. Mutate a local accumulator you own instead. A recursive implementation would also overflow the stack long before 1M.

### Q19. What about `new Array(1e9)` with almost no elements?

`len` is 1e9, so we still iterate 1e9 indices doing `k in O`. Native has the same O(length) behavior. For truly sparse data, use an object/Map and reduce over `Object.keys`, or iterate only present keys.

## Trade-offs / "why not X"

### Q20. Why not just use `for...of` or `forEach` inside your implementation?

`for...of` uses the iterator, which visits holes as `undefined` (wrong for sparse arrays) and doesn't work on plain array-likes without `Symbol.iterator`. `forEach` skips holes correctly but doesn't let us control the starting index for the unseeded case cleanly, and it would be a polyfill depending on another built-in.

### Q21. Why `>>> 0` for length?

It converts `length` to a non-negative 32-bit integer in one step: `undefined → 0`, `"3" → 3`, `-1 → 4294967295`, `NaN → 0`. It matches the older ES5 `ToUint32`; ES2015+ uses `ToLength` (clamp to 0…2^53−1). For real arrays both agree.

### Q22. Your error messages differ from Chrome's. Does that matter?

Tests and callers should depend on the error **type** (`TypeError`), not on engine-specific message text, which differs between V8, SpiderMonkey and JavaScriptCore anyway. That's why the test harness compares `e.constructor.name`.

### Q23. How did you make sure it really matches native?

Every use case runs through both `myReduce` and `reduce` and the results (or error types) are compared, instead of hand-writing expected values. With more time I'd add property-based tests: random sparse arrays, random seeds, compared against native.